Akshay Pranav - Project Portfolio Page

Overview

ItemTasker is a CLI-based Stock Keeping Unit (SKU) Ticketing System designed to manage warehouse inventory and track operational tasks across individual SKUs from the terminal.

Summary of Contributions

Code contributed

RepoSense link

Enhancements Implemented

Edit SKU

Implemented `SKUCommandHandler#handleEditSku()` and `SKU#setLocation()` to relocate SKUs across grid cells without losing attached tasks (Moderate Difficulty). Uses in-place mutation to preserve all task associations, with null-checks, case-insensitive lookup, and `Location` enum validation.

Edit Task

Implemented `TaskCommandHandler#handleEditTask()` and `SKUTaskList#editSKUTask()` for partial task updates, only fields explicitly provided (`d/`, `p/`, `t/`) are modified, preserving the rest (High Difficulty). Backed by a 7-stage validation chain with early-return error handling at each stage.

Sort Tasks

Designed and implemented `TaskSorter.java` as a standalone, SRP-compliant class to sort a SKU's task list by `date`, `priority`, or `status` in ascending or descending order (Moderate Difficulty). Operates on a defensive copy and builds a type-safe `Comparator` via a switch, with field validation before instantiation.

DateValidator

Implemented `DateValidator.java` as a shared static utility for `YYYY-MM-DD` date validation, reused across `addskutask`, `edittask`, and other handlers (Low Difficulty). Validates via regex then `LocalDate.parse()` to catch invalid strings and impossible dates like `2026-02-30`, with a non-blocking warning for past dates.

Mark / Unmark Task

Implemented `TaskCommandHandler#handleMarkTask()` and `handleUnmarkTask()` with duplicate state checks where re-marking a completed task returns an informational message instead of a misleading success message (Moderate Difficulty). Both handlers share the same validation structure, with assertion guards in `SKUTask#mark()` / `unmark()` acting as a secondary safety check.

Contributions to the User Guide (UG)

Authored a FAQ entry on multi-field task editing via `edittask`, and compiled the Command Summary table covering all 16 commands with formats and examples.

Contributions to the Developer Guide (DG)

Architecture

Wrote the Architecture section describing the overall component structure (UI → Parser → CommandRunner → Handlers → Model → Storage) with the accompanying architecture diagram.

Edit SKU / Edit Task

Authored the Edit SKU and Edit Task implementation sections, covering step-by-step scenario walkthroughs, sequence and architecture diagrams, and design considerations such as in-place mutation vs delete-and-recreate.

Contributions to UML diagrams

`architecture.puml`, `edit-sku-sequence.puml`, `edit-sku-architecture.puml`, `edit-task-sequence.puml`, `edit-task-architecture.puml`, `markTaskSequence.puml`, `mark-unmark-architecture.puml`, `unmarkTaskSequence.puml`, `sort-task-sequence.puml`, `sort-task-architecture.puml`

Contributions to team-based tasks

1. **CommandRunner Refactor:** Split the monolithic `CommandRunner` into `TaskCommandHandler`, `SKUCommandHandler`, and `ViewCommandHandler`, applying SRP to improve maintainability for the whole team.
2. **UML Diagram Standardisation:** Overhauled all PlantUML diagrams across the DG by introducing a consistent `style.puml` base, fixing formatting inconsistencies and improving overall readability.
3. **Team Coordination:** Facilitated discussions on architecture and feature design, identified bugs in teammates' code, and assisted in resolving them across the codebase.

Contributions Beyond the Project Team

Peer Quality Assurance (PE-D): Tested a peer team's application during the Practical Exam Dry Run, ranking in the top 10% of bug reporters by identifying critical functionality flaws and documentation bugs.

Contributions to the User Guide (Extracts)

FAQ

Q: Can I edit multiple fields of a task at once? **A**: Yes. The `edittask` command accepts any combination of `d/` (due date), `p/` (priority), and `t/` (description) flags simultaneously. Only the fields you specify will be updated. All other fields are preserved.

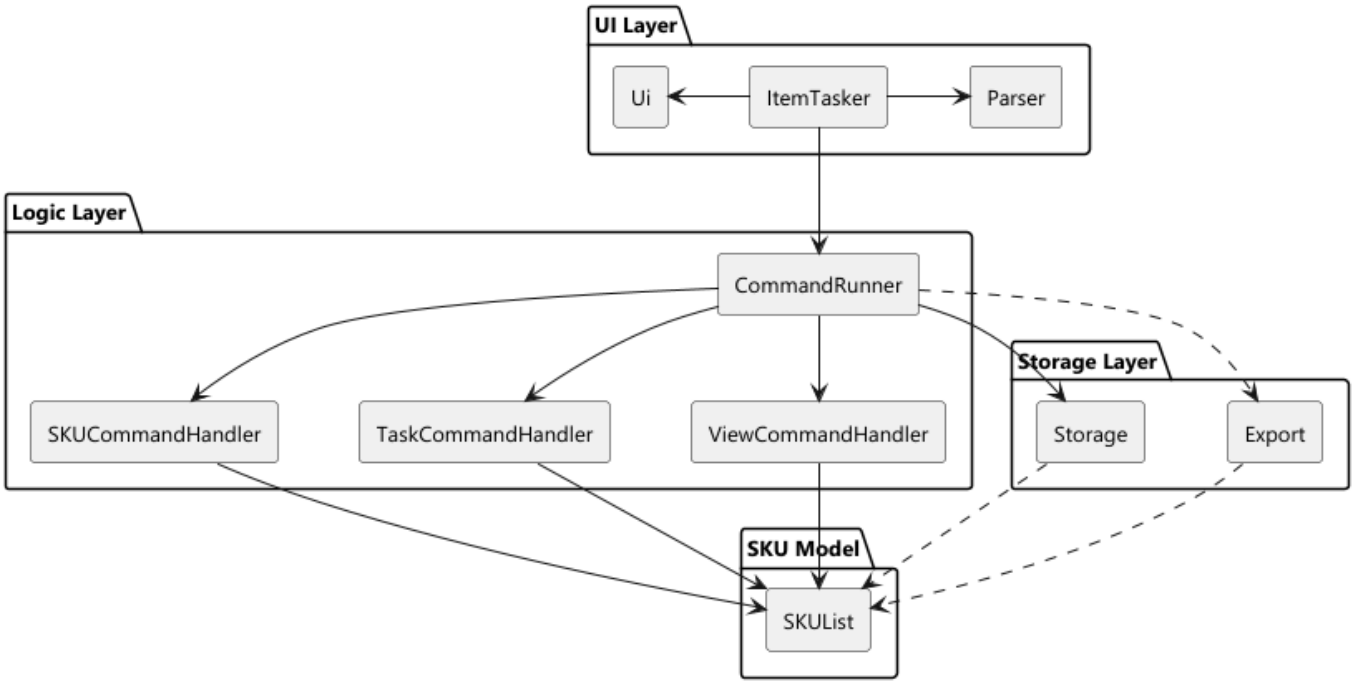
Command Summary

Action	Format	Example
Add SKU	`addsku n/SKU_ID l/LOCATION`	`addsku n/PALLET-A l/B2`
Edit SKU	`editsku n/SKU_ID l/NEW_LOCATION`	`editsku n/PALLET-A l/C3`
Delete SKU	`deletesku n/SKU_ID`	`deletesku n/PALLET-A`
Add task	`addskutask n/SKU_ID d/DUE_DATE [p/PRIORITY] [t/DESCRIPTION]`	`addskutask n/PALLET-A d/2026-06-15 p/HIGH t/Check stock`
Edit task	`edittask n/SKU_ID i/TASK_INDEX [d/DUE_DATE] [p/PRIORITY] [t/DESCRIPTION]`	`edittask n/PALLET-A i/1 d/2026-12-31 p/LOW`
Delete task	`deletetask n/SKU_ID i/TASK_INDEX`	`deletetask n/PALLET-A i/2`
Mark task	`marktask n/SKU_ID i/TASK_INDEX`	`marktask n/PALLET-A i/1`
Unmark task	`unmarktask n/SKU_ID i/TASK_INDEX`	`unmarktask n/PALLET-A i/1`
Sort tasks	`sorttasks n/SKU_ID s/SORT_FIELD [o/ORDER]`	`sorttasks n/PALLET-A s/priority o/descending`
List tasks	`listtasks [n/SKU_ID] [p/PRIORITY] [l/LOCATION]`	`listtasks p/HIGH`
Find tasks	`find [n/SKU_ID] [t/DESCRIPTION] [i/TASK_INDEX]`	`find n/PALLET-A t/Check`
View status	`status [n/SKU_ID]`	`status n/PALLET-A`
View warehouse map	`viewmap`	`viewmap`
Export inventory	`export`	`export`
Help	`help`	`help`
Exit	`bye` / `exit`	`bye`

Contributions to the Developer Guide (Extracts)

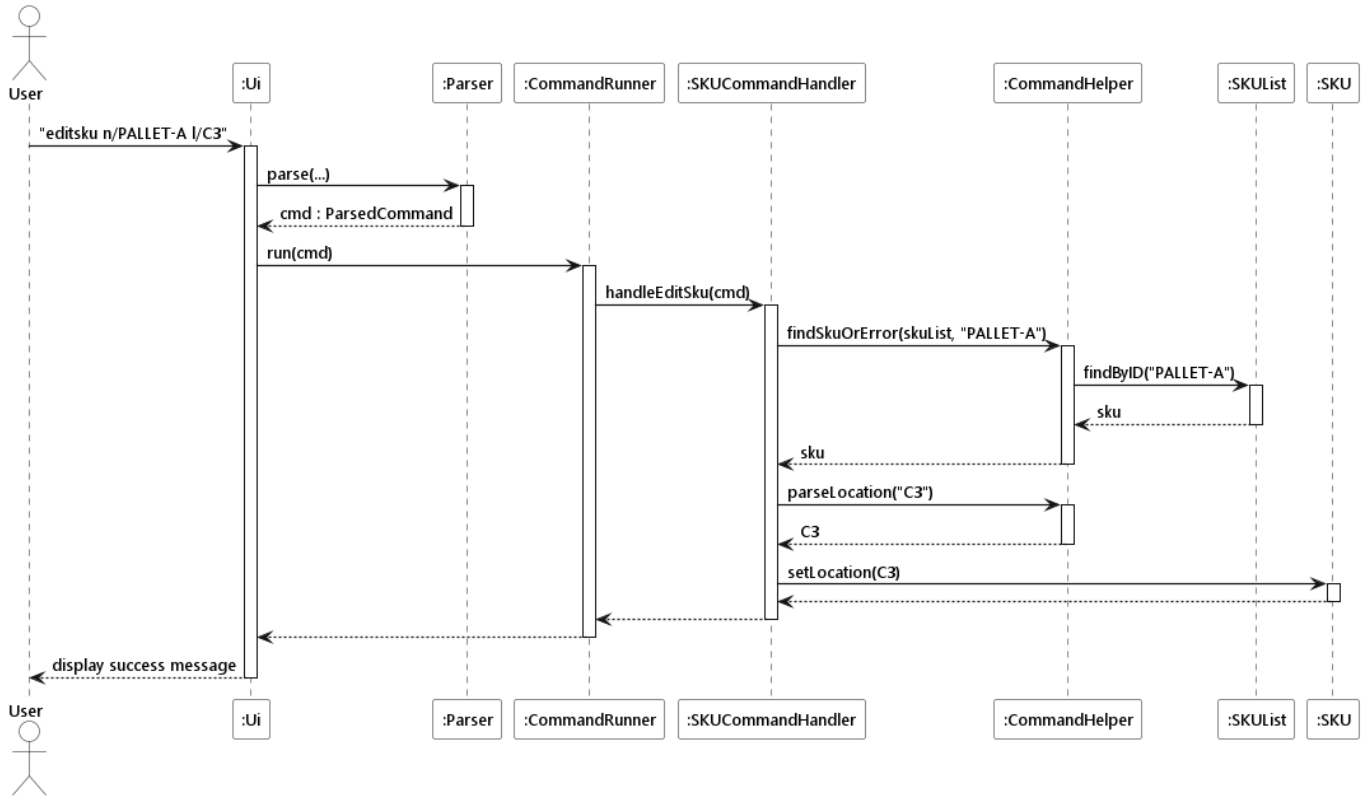
Note that only a small portion is made available here due to page limits.

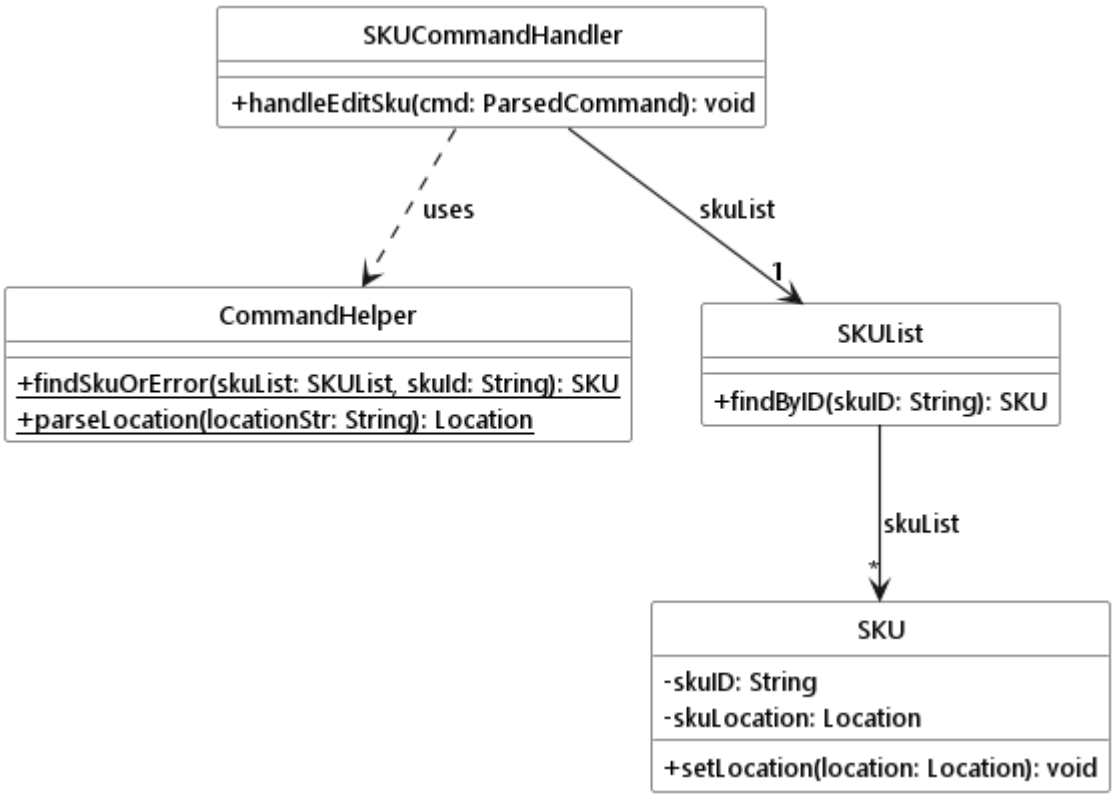
Design: Architecture



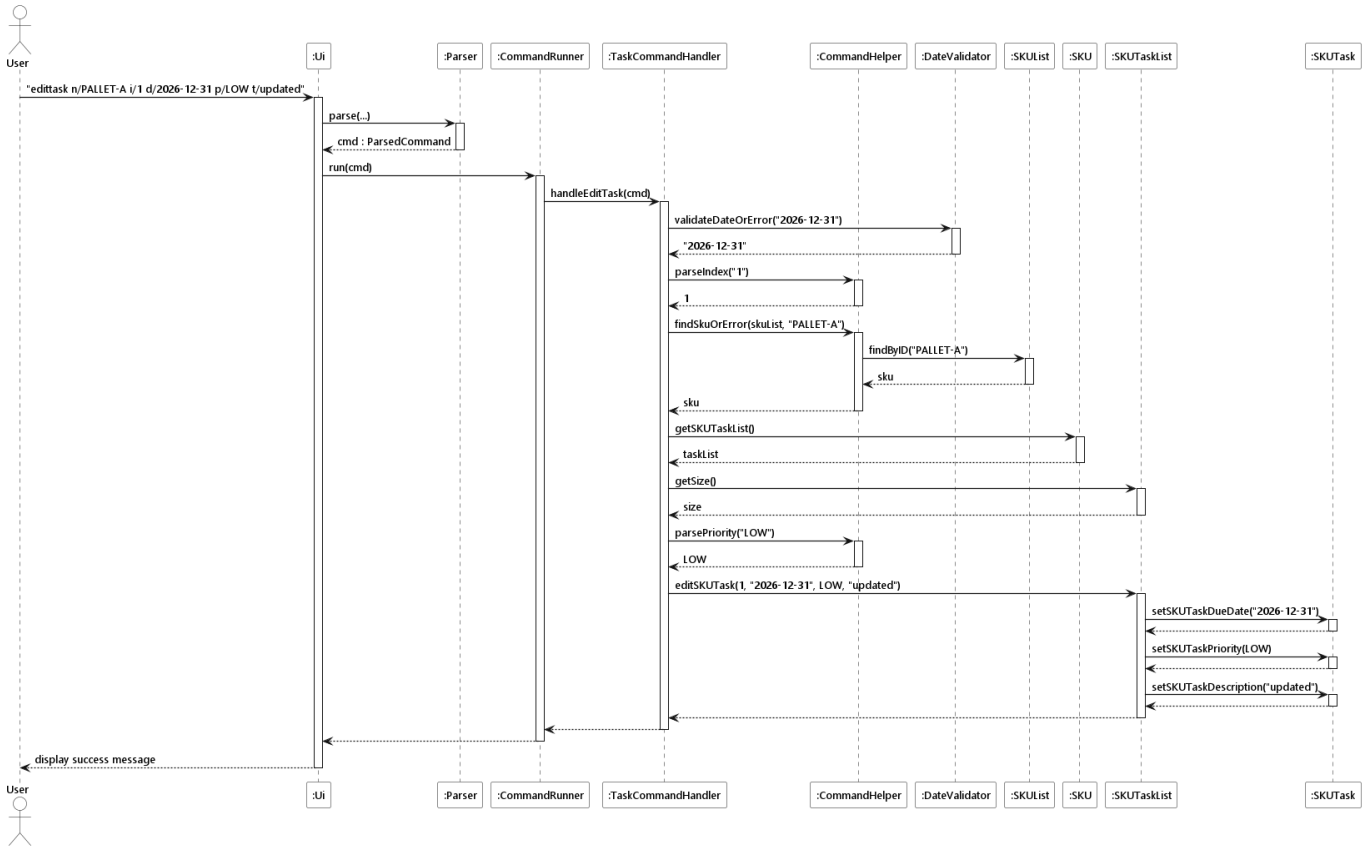
Edit SKU / Edit Task Feature

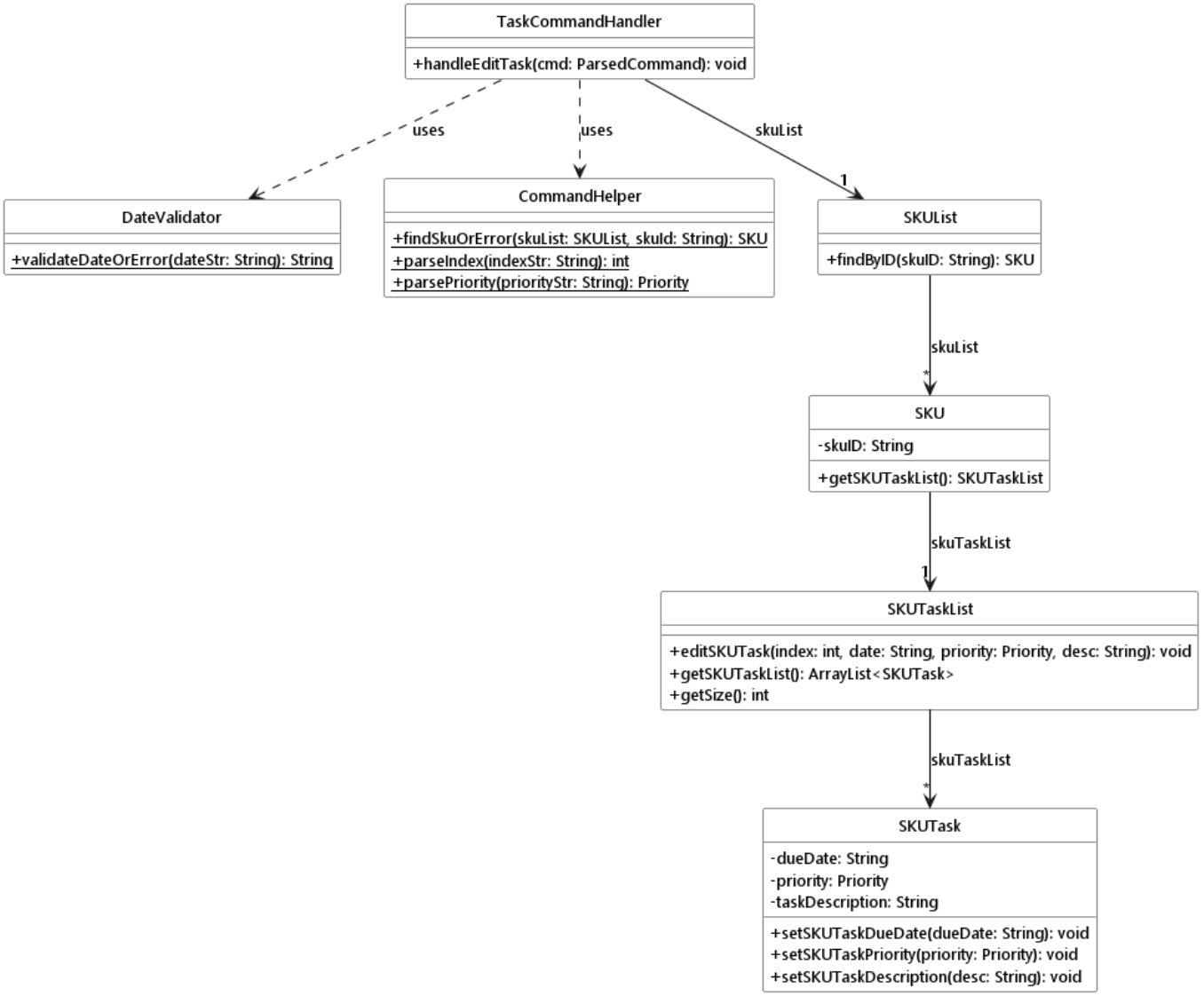
Edit SKU





Edit Task


User
:Ui
:Parser
:CommandRunner
:TaskCommandHandler
:CommandHelper
:DateValidator
:SKUList
:SKU
:SKUTaskList
:SKUTask
"edittask n/PALLET-A i/1 d/2026-12-31 p/LOW t/updated"
parse(...)
cmd : ParsedCommand
run(cmd)
handleEditTask(cmd)
validateDateOrError("2026-12-31")
"2026-12-31"
parseIndex("1")
1
findSkuOrError(skuList, "PALLET-A")
findByID("PALLET-A")
sku
sku
getSKUTaskList()
taskList
getSize()
size
parsePriority("LOW")
LOW
editSKUTask(1, "2026-12-31", LOW, "updated")
setSKUTaskDueDate("2026-12-31")
setSKUTaskPriority(LOW)
setSKUTaskDescription("updated")
display success message


TaskCommandHandler
+handleEditTask(cmd: ParsedCommand): void
uses
uses
skuList
1
DateValidator
+validateDateOrError(dateStr: String): String
CommandHelper
+findSkuOrError(skuList: SKUList, skuId: String): SKU
+parseIndex(indexStr: String): int
+parsePriority(priorityStr: String): Priority
SKUList
+findByID(skuID: String): SKU
skuList
*
SKU
-skuID: String
+getSKUTaskList(): SKUTaskList
skuTaskList
1
SKUTaskList
+editSKUTask(index: int, date: String, priority: Priority, desc: String): void
+getSKUTaskList(): ArrayList<SKUTask>
+getSize(): int
skuTaskList
*
SKUTask
-dueDate: String
-priority: Priority
-taskDescription: String
+setSKUTaskDueDate(dueDate: String): void
+setSKUTaskPriority(priority: Priority): void
+setSKUTaskDescription(desc: String): void